



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه پنجم – تقسیم و حل: مقدمه

دکتر شهرام گلزاری

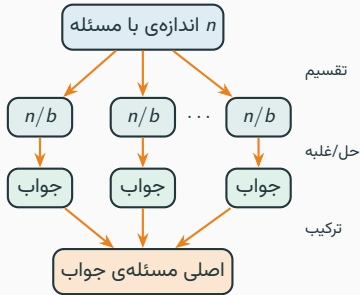
پاییز ۱۴۰۵

در پایان این جلسه انتظار داریم که:

- ایده‌ی روش **تقسیم و حل** و سه گام آن را بشناسیم.
- برای یک تابع بازگشتی **درخت بازگشت** را رسم کنیم و از روی آن تعداد فراخوانی‌ها را بشماریم.
- اجرای یک تابع بازگشتی را قدم به قدم **تریس** کنیم.
- بدانیم در هر لحظه دقیقاً چه چیزی در **پشته‌ی فراخوانی** قرار دارد.

مفهوم تقسيم و حل

سه گام تقسیم و حل (Divide and Conquer)



۱. **تقسیم (divide):** مسئله را به چند زیرمسئله از همان نوع و با اندازه‌ی کوچکتر بشکن.

۲. **حل/غلبه (conquer):** زیرمسئله‌ها را به صورت بازگشتی حل کن، اگر به قدر کافی کوچک‌اند (حالت پایه)، مستقیم حل کن.

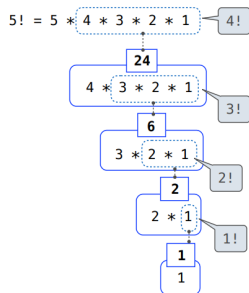
۳. **ترکیب (combine):** از جواب زیرمسئله‌ها جواب مسئله‌ی اصلی را بساز.

بر اساس CLRS، صفحه ۷۶

```
1 static long fact(int n) {  
2     if (n == 0 || n == 1)  
3         return 1;  
4     else  
5         return n * fact(n - 1);  
6 }
```

- حالت پایه: خط‌های ۲ و ۳.
- حالت بازگشتی: خط ۵؛ مسئله‌ی $n!$ به مسئله‌ی کوچک‌تر $(n-1)!$ تبدیل می‌شود.

Factorial



```
def fact(n):
    """Compute n factorial.

    >>> fact(5)
    120
    >>> fact(0)
    1
    """
    if n == 0 or n == 1:
        return 1
    else:
        return n * fact(n-1)
```

Figure 11.10: Recursive calculation of 5!.

7

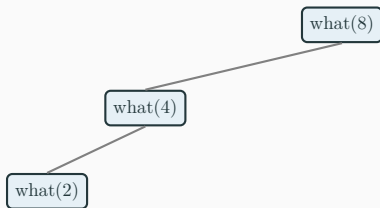
یک مثال

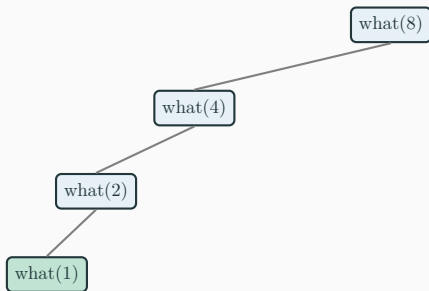
- حالت پایه: خط‌های ۲ و ۳.
- حالت بازگشتی: خط ۴؛ دو فراخوانی با اندازه‌ی $n/2$.
- پرسش: $\text{what}(8)$ روی هم چند فراخوانی انجام می‌دهد؟

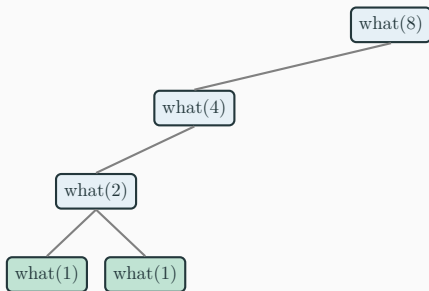
```
1 static int what(int n) {  
2     if (n <= 1)  
3         return 1;  
4     return what(n / 2) + what(n / 2);  
5 }
```

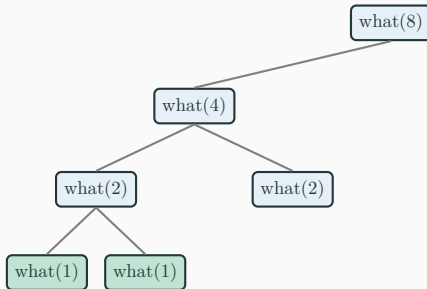
what(8)

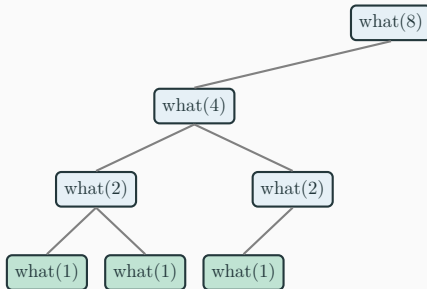


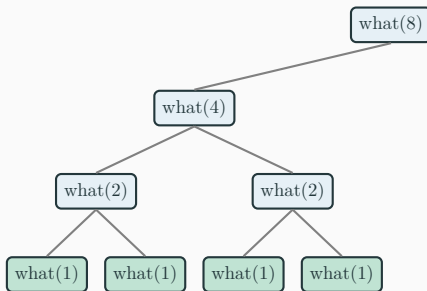


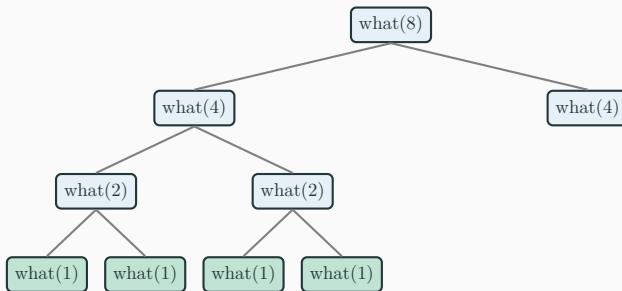


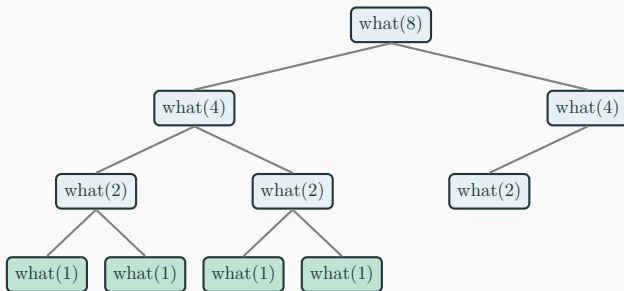


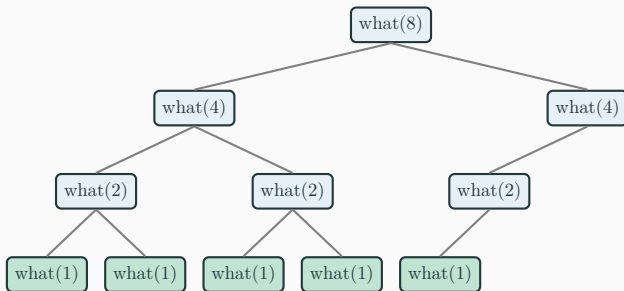


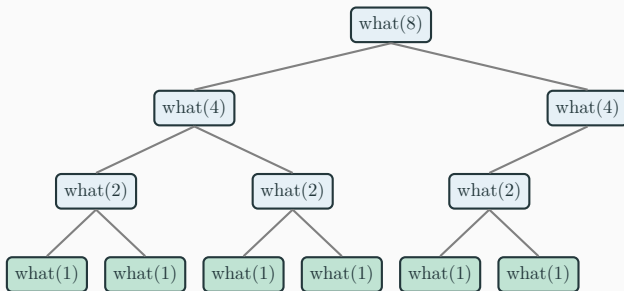


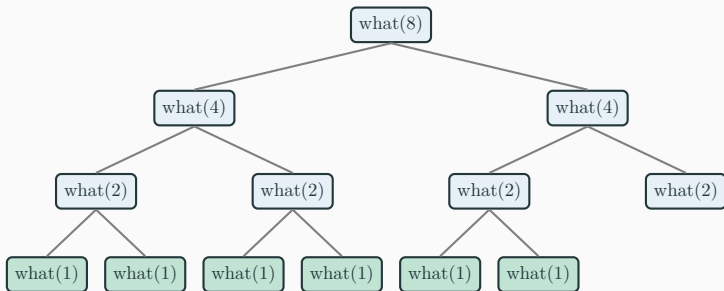


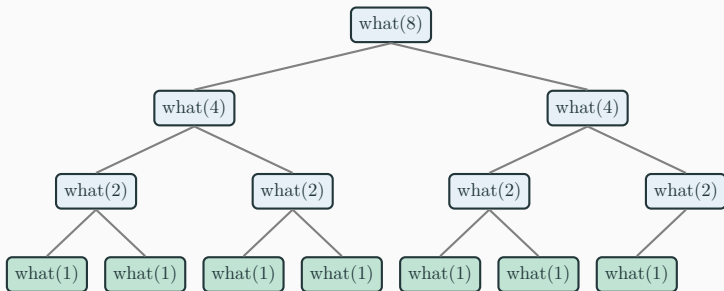


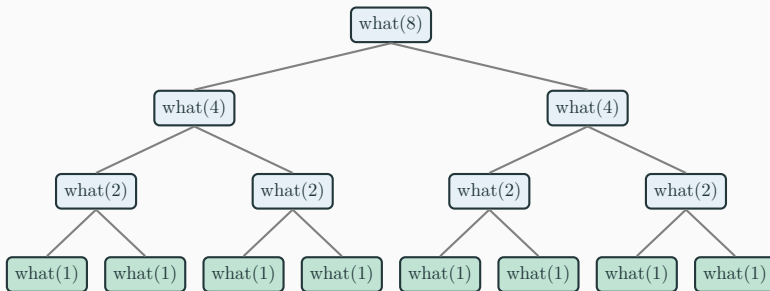












هر گره یک فراخوانی است. هر گرهی غیربرگ **دو فرزند** دارد، چون خط ۴ دو بار $\text{what}(n / 2)$ را صدا می‌زند. برگ‌های سبز حالت پایه‌اند. گره‌ها به همان ترتیبی ظاهر می‌شوند که فراخوانی‌ها اجرا می‌شوند: زیردرخت چپ کامل می‌شود و بعد نوبت زیردرخت راست است.

جمع: $۱۵ = ۱ + ۲ + ۴ + ۸$ فراخوانی.

یک تغییر کوچک در what

- فقط خط ۴ تغییر کرده است: به جای جمع دو فراخوانی، $\text{what}(n / 2)$ یک بار صدا زده می‌شود و جوابش در ۲ ضرب می‌شود.
- پرسش ۱: خروجی این نسخه با نسخه‌ی قبل یکسان است؟
- پرسش ۲: $\text{what}(8)$ حالا چند فراخوانی انجام می‌دهد؟

```
1 static int what(int n) {  
2     if (n <= 1)  
3         return 1;  
4     return 2 * what(n / 2);  
5 }
```

نسخه‌ی جدید برای هر n دقیقاً همان مقدار نسخه‌ی قبل را برمی‌گرداند:

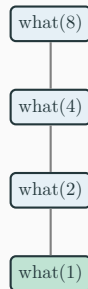
۸	۷	۶	۵	۴	۳	۲	۱	n
۸	۴	۴	۴	۴	۲	۲	۱	what(n)

- اما خروجی یکسان به معنای تعداد فراخوانی یکسان نیست. در نسخه‌ی قبل، $\text{what}(n / 2)$ دو بار جداگانه حساب می‌شد.

پرسش ۲: درخت بازگشت نسخه‌ی جدید ($n = 8$)

هر گره فقط **یک فرزند** دارد: $\text{what}(n / 2) * 2$ فقط یک فراخوانی است؛ ضرب در ۲ بعد از برگشتن جواب انجام می‌شود.

نسخه‌ی جدید	نسخه‌ی قبل	($n = 8$)
۴	۱۵	تعداد گره‌ها (فراخوانی‌ها)
۴	۴	تعداد سطح‌های درخت



پشته و پشته‌ی فراخوانی

«پشته یکی از ساده‌ترین داده‌ساختارها در علوم کامپیوتر است. پشته هم مانند لیست چند مقدار را نگه می‌دارد، اما برخلاف لیست، اضافه کردن یا برداشتن مقادارها را فقط به “بالای” پشته محدود می‌کند. در پشته‌هایی که با لیست یا آرایه پیاده‌سازی می‌شوند، “بالا” همان آخرین عنصر است، یعنی انتهای سمت راست لیست یا آرایه. به اضافه کردن مقدار، **درج** (push) مقدار در پشته و به برداشتن مقدار، **حذف** (pop) مقدار از پشته می‌گویند.»

Al Sweigart, The Recursive Book of Recursion، صفحه ۷ (ترجمه)

«فرض کنید با کسی گفت‌وگویی پرپیچ‌وخم دارید. درباره‌ی دوستتان آلیس حرف می‌زنید و این شما را یاد داستانی درباره‌ی همکارتان باب می‌اندازد، اما برای اینکه آن داستان معنا داشته باشد، اول باید چیزی درباره‌ی دخترعمویتان کارول توضیح دهید. داستان کارول را تمام می‌کنید و به حرف زدن درباره‌ی باب برمی‌گردید، و وقتی داستان باب تمام شد، به حرف زدن درباره‌ی آلیس برمی‌گردید. بعد یاد برادرتان دیوید می‌افتید و داستانی درباره‌ی او تعریف می‌کنید. سرانجام داستان اصلی‌تان درباره‌ی آلیس را تمام می‌کنید.»

صفحه‌های ۷ و ۸



«ساختار گفت‌وگوی شما، مانند شکل ۱-۵، شبیه پشته است. این گفت‌وگو شبیه پشته است، چون موضوع فعلی همیشه در بالای پشته قرار دارد.»

Al Sweigart, The Recursive Book of Recursion، صفحه ۸ (ترجمه؛ بازترسیم شکل ۱-۵)

«برنامه‌ها هم از پشته استفاده می‌کنند. **پشته‌ی فراخوانی** برنامه، که به آن فقط **پشته** هم می‌گویند، پشته‌ای از اشیای قاب است. **اشیای قاب**، که به آن‌ها فقط **قاب** هم می‌گویند، اطلاعات یک فراخوانی تابع را در خود دارند، از جمله اینکه کدام خط کد تابع را صدا زده است، تا وقتی تابع برمی‌گردد اجرا بتواند به همان‌جا برگردد.»

صفحه ۹

«اشیای قاب هنگام فراخوانی تابع ساخته و در پشته درج می‌شوند. وقتی تابع برمی‌گردد، آن شیء قاب از پشته حذف می‌شود. اگر تابعی را صدا بزنیم که تابعی را صدا می‌زند که آن هم تابعی را صدا می‌زند، پشته‌ی فراخوانی سه شیء قاب خواهد داشت. وقتی همه‌ی این توابع برگردند، هیچ شیء قابی روی پشته‌ی فراخوانی نخواهد بود.»

صفحه ۱۰

(ترجمه) *Al Sweigart, The Recursive Book of Recursion*

«برنامه‌نویس‌ها لازم نیست برای کار با اشیای قاب کدی بنویسند، چون زبان برنامه‌نویسی این کار را خودکار انجام می‌دهد. زبان‌های برنامه‌نویسی مختلف اشیای قاب را به روش‌های متفاوتی پیاده‌سازی می‌کنند، اما به‌طور کلی قاب‌ها این موارد را در خود دارند:

- **نشانی بازگشت**، یعنی جایی از برنامه که اجرا باید پس از برگشتن تابع به آن‌جا برود
- آرگومان‌هایی که به تابع فرستاده شده‌اند
- مجموعه‌ای از متغیرهای محلی که هنگام فراخوانی تابع ساخته می‌شوند»

Al Sweigart, The Recursive Book of Recursion، صفحه ۱۰ (ترجمه)

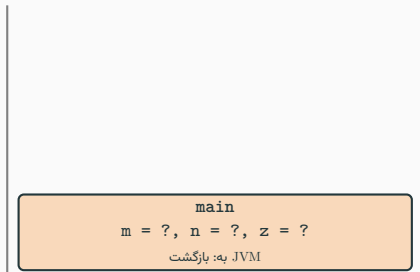
تريس فراخوانی توابع

- main تابع avg را صدا می‌زند و avg تابع sum را.
• ورودی: $m = 4$ و $n = 3$.

```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

- شمارنده‌ی برنامه (Program Counter, PC): نشانی دستوری را نگه می‌دارد که باید اجرا شود.
- فراخوانی (call): یک قاب تازه روی پشته push می‌شود. پارامترها، متغیرهای محلی و **نشانی بازگشت** (دستوری که بعد از تمام شدن تابع باید اجرا شود) در این قاب ذخیره می‌شوند.
- بازگشت (return یا {}): scope تابع تمام می‌شود و قابش pop می‌شود.

پشته بالای



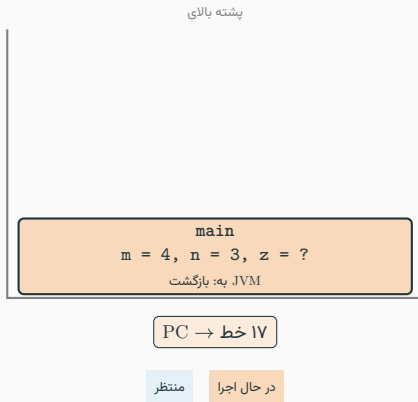
PC → خط ۱۵

منتظر

در حال اجرا

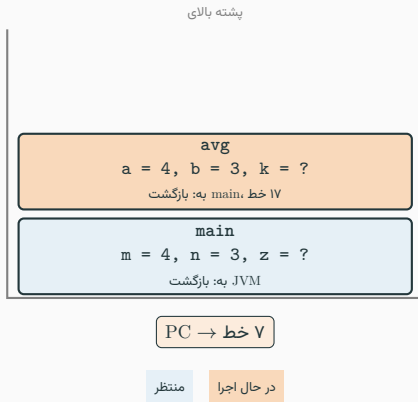
```
1 static int sum(int b, int a) {
2     int d;
3     d = b + a;
4     return d;
5 }
6 static double avg(int a, int b) {
7     double k;
8     k = sum(a, b) / 2.0;
9     return k;
10 }
11 public static void main(String[] args) {
12     Scanner in = new Scanner(System.in);
13     int m, n;
14     double z;
15     m = in.nextInt();    // 4
16     n = in.nextInt();    // 3
17     z = avg(m, n);
18     System.out.println(z);
19 }
```

گام ۱ از ۱۰: اجرای برنامه از main شروع می‌شود و قاب main روی پشته push می‌شود. متغیرهای m, n و z در این قاب جا دارند ولی هنوز مقدار ندارند.



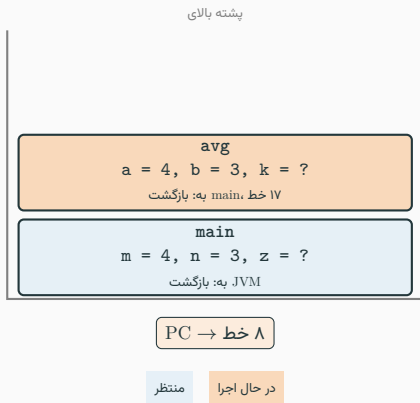
```
1 static int sum(int b, int a) {
2     int d;
3     d = b + a;
4     return d;
5 }
6 static double avg(int a, int b) {
7     double k;
8     k = sum(a, b) / 2.0;
9     return k;
10 }
11 public static void main(String[] args) {
12     Scanner in = new Scanner(System.in);
13     int m, n;
14     double z;
15     m = in.nextInt();    // 4
16     n = in.nextInt();    // 3
17     z = avg(m, n);
18     System.out.println(z);
19 }
```

گام ۲ از ۱۰: ورودی‌ها خوانده شدند: $m = 4$ و $n = 3$. PC حالا روی خط ۱۷ است که `avg(m, n)` را صدا می‌زند.



```
1 static int sum(int b, int a) {
2     int d;
3     d = b + a;
4     return d;
5 }
6 static double avg(int a, int b) {
7     double k;
8     k = sum(a, b) / 2.0;
9     return k;
10 }
11 public static void main(String[] args) {
12     Scanner in = new Scanner(System.in);
13     int m, n;
14     double z;
15     m = in.nextInt();    // 4
16     n = in.nextInt();    // 3
17     z = avg(m, n);
18     System.out.println(z);
19 }
```

گام ۳ از ۱۰: قاب avg ساخته و push می‌شود. مقدار m و n در a و b کپی می‌شود. **نشانی بازگشت** (خط ۱۷ در main) در قاب avg ذخیره می‌شود و PC به بدنه‌ی avg می‌رود.

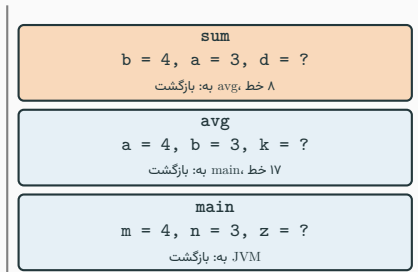


```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

گام ۴ از ۱۰: خط ۸ برای محاسبه‌ی k به مقدار sum(a, b) نیاز دارد، پس sum را صدا می‌زند. avg تا برگشتن جواب منتظر می‌ماند.

تریس فراخوانی‌ها و پشت‌پشتی فراخوانی

پشته بالای



۲ خط → PC

منتظر

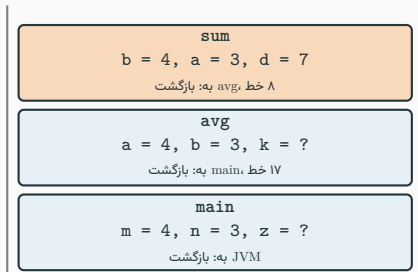
در حال اجرا

```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

گام ۵ از ۱۰: قاب sum push می‌شود. دقت کنید: پارامترها به ترتیب جایگاه مقدار می‌گیرند، نه بر اساس نام. b در sum مقدار a در avg (یعنی ۴) را می‌گیرد و a در sum مقدار ۳ را.

تریس فراخوانی‌ها و پشت‌پشتی فراخوانی

پشته بالای



۴ خط → PC

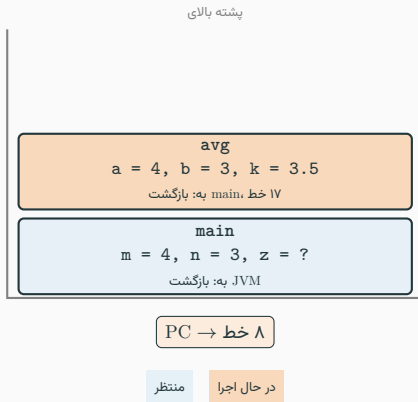
منتظر

در حال اجرا

```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

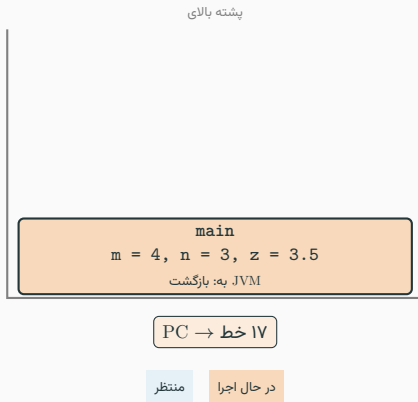
گام ۶ از ۱۰: خط ۳: $d = b + a = 7$. حالا PC روی `return d` است. سه قاب هم‌زمان روی پشت‌پشتی؛ فقط قاب بالایی (sum) در حال اجراست.

تریس فراخوانی‌ها و پشته‌ی فراخوانی



```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

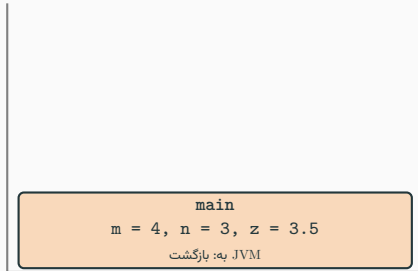
گام ۷ از ۱۰: با `return`، scope تابع `sum` تمام می‌شود و قابش `pop` می‌شود. **نشانی بازگشت** ذخیره‌شده در آن قاب در `PC` قرار می‌گیرد و اجرا از خط ۸ ادامه می‌یابد: $k = 7 / 2.0 = 3.5$.



```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

گام ۸ از ۱۰: مقدار k را برمی‌گرداند و قابش pop می‌شود؛ a، b و k دیگر وجود ندارند. PC به خط ۱۷ در main برمی‌گردد و z = 3.5.

پشته بالای



PC → خط ۱۸

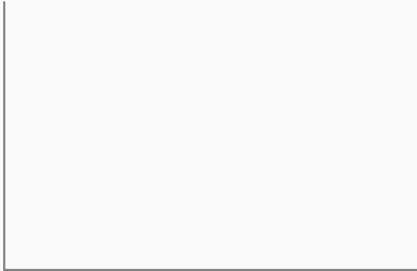
منتظر

در حال اجرا

```
1 static int sum(int b, int a) {  
2     int d;  
3     d = b + a;  
4     return d;  
5 }  
6 static double avg(int a, int b) {  
7     double k;  
8     k = sum(a, b) / 2.0;  
9     return k;  
10 }  
11 public static void main(String[] args) {  
12     Scanner in = new Scanner(System.in);  
13     int m, n;  
14     double z;  
15     m = in.nextInt();    // 4  
16     n = in.nextInt();    // 3  
17     z = avg(m, n);  
18     System.out.println(z);  
19 }
```

گام ۹ از ۱۰: خط ۱۸ مقدار 3.5 را چاپ می‌کند.

پشته بالای



است خالی پشته

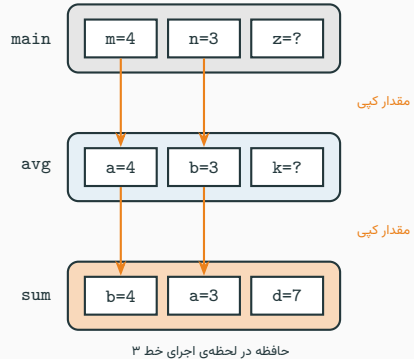
منتظر

در حال اجرا

```
1 static int sum(int b, int a) {
2     int d;
3     d = b + a;
4     return d;
5 }
6 static double avg(int a, int b) {
7     double k;
8     k = sum(a, b) / 2.0;
9     return k;
10 }
11 public static void main(String[] args) {
12     Scanner in = new Scanner(System.in);
13     int m, n;
14     double z;
15     m = in.nextInt();    // 4
16     n = in.nextInt();    // 3
17     z = avg(m, n);
18     System.out.println(z);
19 }
```

گام ۱۰ از ۱۰: main هم تمام می‌شود و قابش pop می‌شود. پشته خالی است.

- هر فراخوانی متغیرهای خودش را دارد؛ a در avg و a در sum دو خانه‌ی جدا هستند.
- بعد از return، حافظه‌ی قاب آزاد می‌شود.



منابع

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition
- Al Sweigart — *The Recursive Book of Recursion*, No Starch Press
- UC Berkeley CS 61A — Recursion lecture